

AUB Card Cashier API Guide for E-Commerce Merchant

[Version No.: V2.2]

Catalog

AUB Card Gateway API Guide

1	File History	3
1.1	Before integrating API	4
2	Integration Introduction.....	4
2.1	Generate merchant public and private key(By using java idea):	4
2.2	Upload merchant key and download AUB institution key.....	6
2.3	Import the project and modify the demo content	7
2.4	Run the demo	8
3	API Introduction.....	9
3.1	Target readers	9
3.2	Requirement.....	9
4	Interface Specification	10
4.1	Interface calling rules	10
4.2	Interface security.....	11
4.3	Public parameters	14
4.4	Interface domain name	15
4.5	Request example	15
5	Cashier Interface	17
5.1	H5 Payment	17
5.2	Query.....	19
5.3	Refund	21
5.4	Notification.....	23
6	Appendix	24
6.1	Error code.....	24

1 File History

Revision Date	Revised Content	Version
2021-04-20	New	V1.1
2021-11-09	Add integration Introduction part	V2.1
2025-05-21	Add missing Notification parameters	V2.2

1.1 Before integrating API

We use the encrypted transmission of JWS and JWE for MC transaction, so we will need to use the RSA key. For the algorithm of JWS and JWE, please refer to chapter 3.2.1 and 3.2.2. This chapter guides you how to generate RSA key and how to use it.

2. Integration Introduction

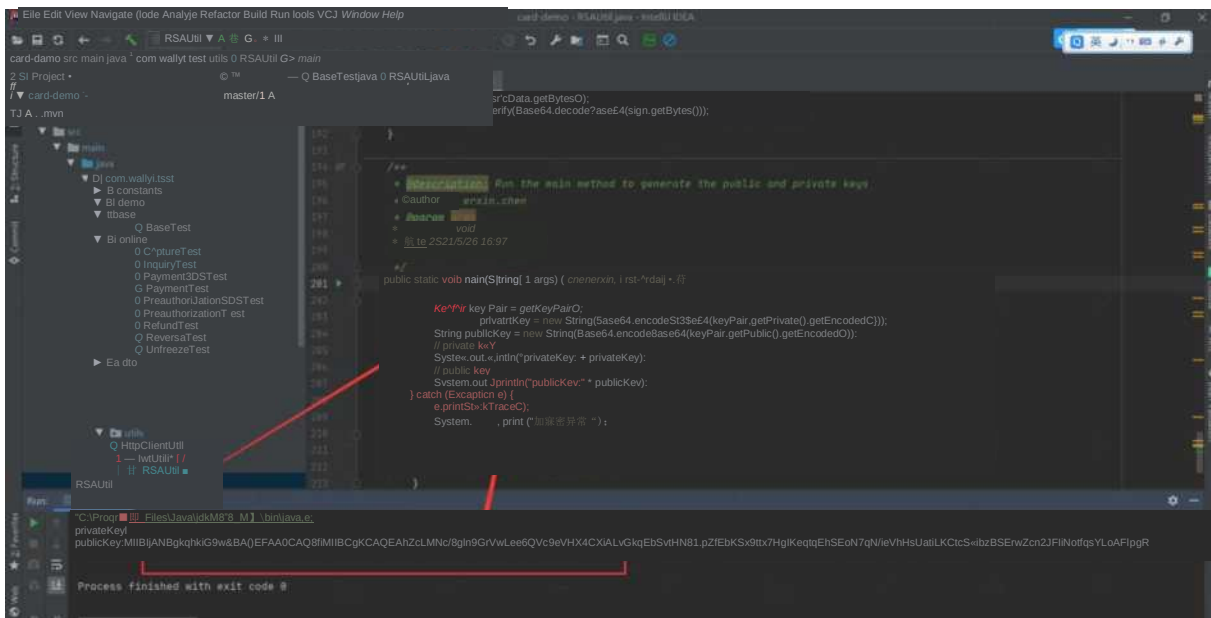
In this part, you need to generate two pairs of keys, one for JWS and another for JWE

2.1 Generate merchant public and private key(By using java idea):

2.1.1 (method 1) Generate merchant public and private key with Java

IDEA

Run **RSUtil** in the demo to generate sign keys (JWS)



2.1.2 Generate merchant public and private key

Repeat step 2.1.1 to generate another pair of encryption key (JWE)

2.1.3 (Method 2) Generate merchant public and private key for those users that do not use Java IDEA

According to the instructions of the interface document, generate public and private keys in the Linux environment (in the Linux environment, copy and paste [shift-insert key], and then enter the following three red codes until the final file is exported, end of key pair generation)

2.1.3.1 Generate RSA private key:

```
openssl genrsa -out rsa_private_key_2048.pem 2048
```

```
[root@hadoop3 ~]# openssl genrsa -out rsa_private_key_2048.pem Generating RSA private key, 1024 bit long modulus
```

```
... ..++++++
e is 65537 (0x10001)
```

2.1.3.2 Convert rsa private key to pkcs encoding

```
openssl pkcs8 -topk8 -in rsa_private_key_2048.pem -out pkcs8_rsa_private_key_2048.pem -nocrypt
```

```
[root@hadoop3 ~]# openssl pkcs8 -topk8 -in rsa_private_key_2048.pem -out pkcs8_rsa_private_key_2048.pem -nocrypt
```

2.1.3.3 Generate public key

```
openssl rsa -in rsa_private_key_2048.pem -out rsa_public_key_2048.pem -pubout
```

```
[root@hadoop3 ~]# openssl rsa -in rsa_private_key_2048.pem -out rsa_public_key_2048.pem -pubout
writing RSA key
```

2.1.3.4 Unpack

```
tar czvf key.tar.gz rsa_private_key_2048.pem rsa_public_key_2048.pem pkcs8_rsa_private_key_2048.pem
```

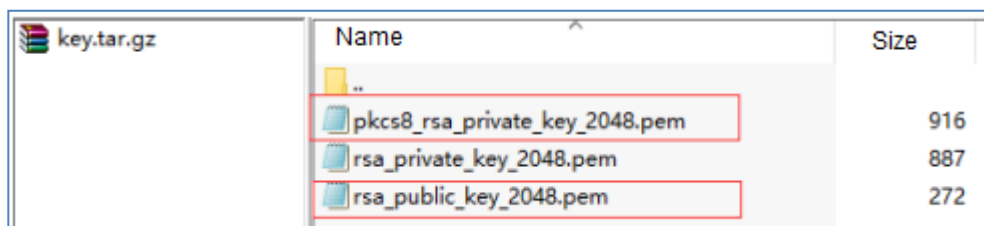
```
[root@hadoop3 ~]# tar czvf key.tar.gz rsa_private_key_2048.pem rsa_public_key_2048.pem pkcs8_rsa_private_key_2048.pem
```

2.1.3.5 Export to local

```
sz key.tar.gz
```

```
[root@hadoop3 ~]# sz key.tar.gz
```

2.1.3.6 After exporting, finally use the public and private keys in the red box in the figure below



Name	Size
pkcs8_rsa_private_key_2048.pem	916
rsa_private_key_2048.pem	887
rsa_public_key_2048.pem	272

2.1.3.7 Remove the blank spaces in the key (if you generate according to 2.2.3 there will be blank spaces)

```

1 -----BEGIN PRIVATE KEY-----
2 MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQC07M5NEVmlQnB
3 0ENtK6F4xbEgFKYsIRdo4zqvrgEAM/UE/asbqRcSuuIv0xH0y3hgC19wadledpL
4 dYfEWR7O/2811RgmjwNh0DOSdaFnev2us1d4Tm/4q5L96yhw18GQq8yBj2bHi13
5 M8ShxWb8s0kZh504pc7RuMcK+tZ+4GtDbJ4AVgy7kCwuPzCTVEfUJyCoHp96epV
6 N/BfxpW22vIsOzcFSXAFcyCh6FgKib7b51S5ke2ow3MLEBz7HIV786NuC4iyFgmQ
7 lq1vo5c1ARuPQLWlywveme0GDmLwKl3kbg4Q/uS/cKeIVj14KyIw0VinMMA2yn4
8 PaUA61WjAgMBAAEcgEAAQn9+6gsvKMJr4HUCM1VHbsvebk/y7f117KLW4P1tXt
9 BK7Lhz09MDXGQjQPaUzQoBccx6X3aX2reffWp1bmj2uoIdLxioS7azIJE7vynt
10 IivndVfyWgIEHxMDXGgFpJazAzFdqCCDviHK66gtRQ1yyRpy9WQNgG8NFz9MSz5
11 i1Jvg/LleigqWksehXMYrcCKXtkfBtVWkoxS6pJlynH8HEFimm/1aDLko05sqdxs
12 iNuGayqLsR01Rp9onP11H6SQ1Fi9fhQEnK+boKAV0nM5TbS+jugtCwighG3+QJa
13 ub874Q5JLcQpN6sfd3bons7Rx962eQvUMxi/ZTVuqKbGQDe9N18T1tHh7LRd3se
14 alpwpTdDWLgYi7w5FwsbyrAar/Axa3lRhKDLfe3ka98B154hvwSe7iILW3sA8WA0
15 flpF0N+0Rs+i8Pmcz9fj2YzDt5IkjG6WaqESj163vV8sxMLC2NXab2jzCEkVv0x
16 NTFc4H6zfTorrNiVND4SUyoIpQHBGQDEVT/tXmRvvTVU3y/BpIeZ+99o01zGxc/4
17 5mSurJlqFyupmPCB02D2Lcm9mDkQASpRaywT1EUKHA0fD6piQC5+kqEe0eEu8t
18 coqh/KSE1ZwcvgDFS/3tno0T7CNqN9S3Yqn29xHSpdRK5IUUdq4EfdUuFnWtY8g
19 bOPopeBKpKBGFeLIp5p9nhmsvMgJCRMNEjIXqM+AcM5kuWdW@vc0TsEXCG7hn5Y
20 q15ciV1G0eB7oMqozpa+N6QA1JpxLlpQfQJKvJvntf3PjBBDMgkfcEyaCPPLosgm
21 iF1FTYysQe0tOp/jwgir+DR9S10rqQUS9Y5G+bDQ/QEQWkarHy64Y01AoGBALEL
22 y2XTogmQj2N2606PmHn1rZu7N0C3h2MLiBdOScJXBncCm9aLOJjLR0TPifg9mFGJ
23 HXUvN7X3OkQJKodJ+Tr4x0DxckjK1Vg5ZQL9ugBAttQ6pPCLqBvnyvK2T/QLTnljE
24 nSmB9hCe//TsGXc9RkXrtchj7T0N83NjIFkICcXVAoGARd4v4j2qvT1400GWRk9
25 NeVRLXmaTaccs1KneNMk6j2HnryMqbtvTeQp5TOSrpDivozxuNVfIbngRdgJ+kXIP
26 r902ZL6mDMKGB7FHSLO9DMF/TmFW01bt3EJ/BA+/VsdC7czJQ4ajIy/bXDOu2C
27 GLKFO89756y2KW0GTmVede4=
28 -----END PRIVATE KEY-----

```

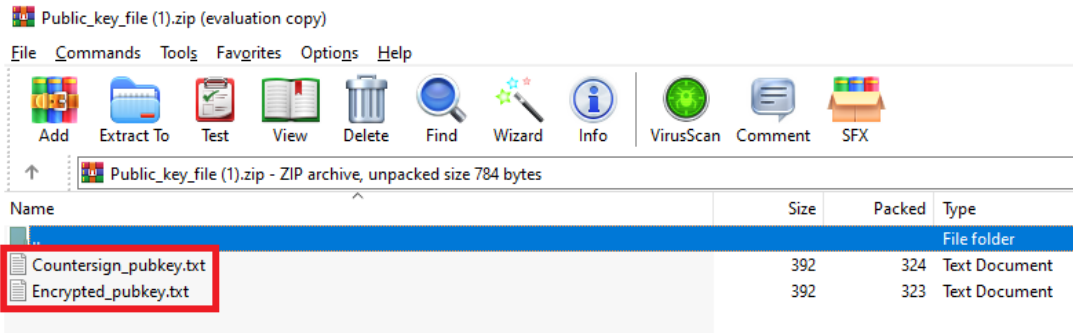
2.2 Upload the merchant key and download the AUB institution key

2.2.1 Upload merchant public key that you've generated in chapter 2.1

Upload sign public key as your JWS key and upload encrypt public key as your JWE key.

2.2.2 Download Institution public key:

Institution public keys will be used for chapter 3.4.3



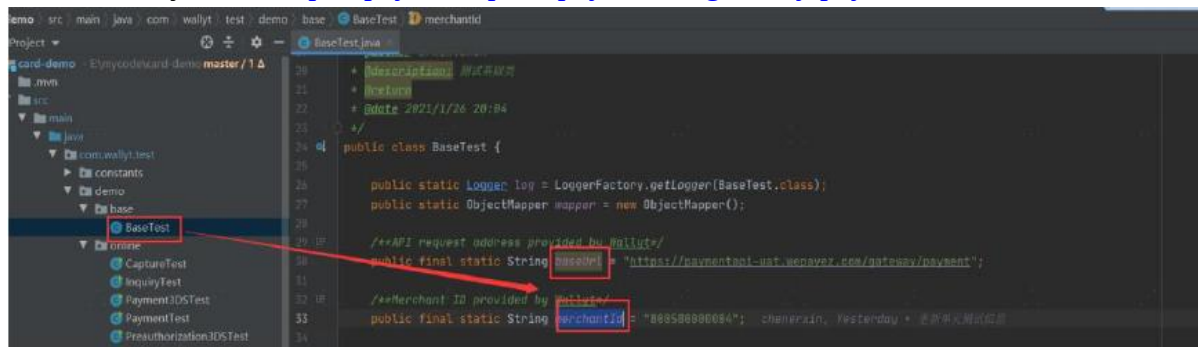
2.3 Import the project and modify the demo content

2.3.1 Import project:

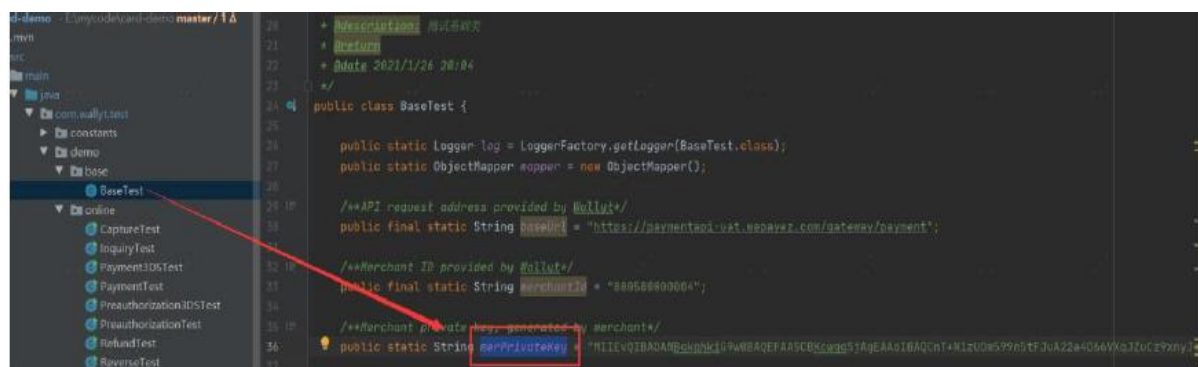
We provide the Java card-demo project under IDEA/eclipse, which can be used for your testing. Demo BaseTest request parameter values modification:

2.3.2 Replace baseUrl and merchantId:

Prod Gateway URL: <https://paymentapi.wepayez.com/gateway/payment>



2.3.3 Replace demo merPrivateKey with merchant JWS private key generated in Chapter 2.1

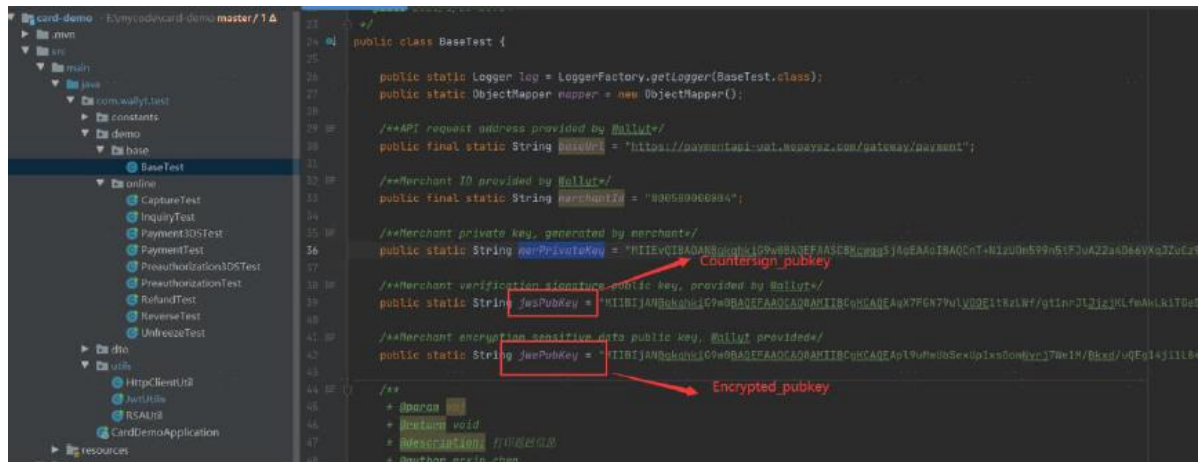


2.3.4 Replace the institution public key with the institution public key downloaded in 2.2.2

AUB Card Gateway API Guide

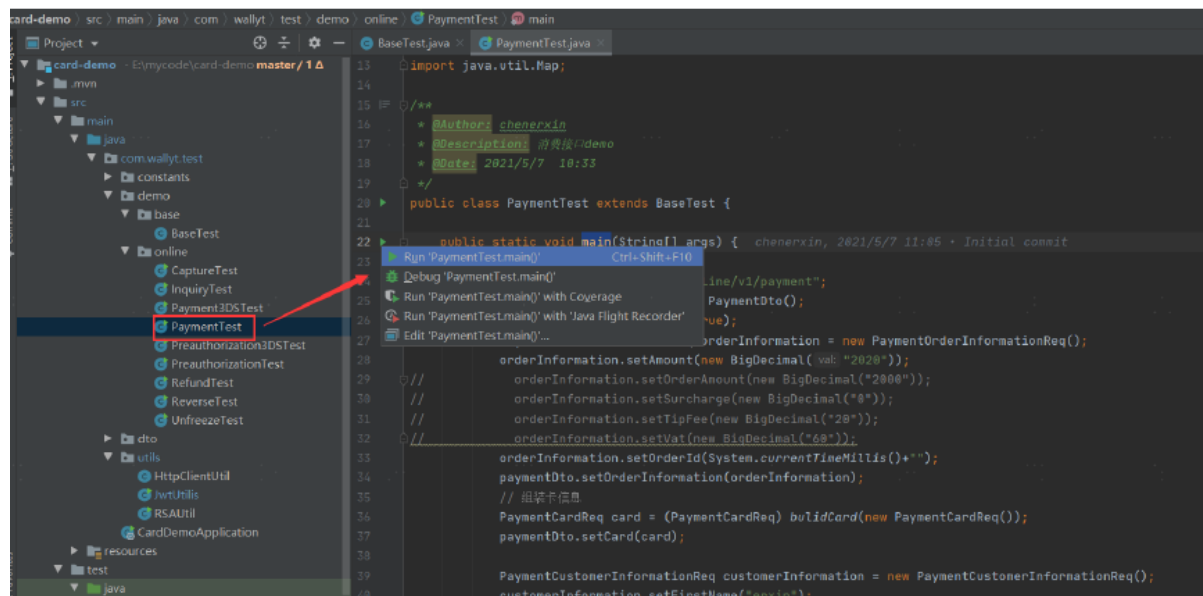
jwsPubKey corresponds to Countersign_pubkey downloaded in 2.2.2

jwePubKey corresponds to Encrypted_pubkey downloaded in 2.2.2



2.4 Run the demo

Take the Cashier Payment interface as an example, the other methods would be the same.



3 API Introduction

3.1 Target readers

People who position in Product Manager, Software Developer, or other related posts that needs to access to our system and do further operations online by APIs.

3.2 Requirement

Option	Full name	Description
M	Mandatory	Items that must be filled in
C	Conditional	Items that should be filled in according to conditions
O	Optional	Optional items

4 Interface Specification

4.1 Interface calling rules

REST APIs are based on HTTP APIs designed in RESTful style. Any API request must use HTTPS. The message bodies of request and response must be in JSON format.

4.1.1 Authentication

The API verifies the identity of the merchant and the integrity of the data by verifying the signature of the request. The merchant needs to use the JWS algorithm to countersign the request content to generate a string. The signature information of the request is passed through the HTTP header Authorization. Please refer to the 3.2.2 JWD section for detail description. Requests that do not carry a signature or fraudulent signature will not be executed. The API will also include the signature information in the successfully processed response.

4.1.2 Data format

REST APIs only use JSON as the data exchange format of message body. The request must set the Content-Type and Accept corresponding to the HTTP header to *application/json*.

4.1.3 Character set

REST API only supports a subset of UTF-8 character encoding: 1-byte~3-byte characters. i.e. REST API does not support 4-byte~6-byte characters in the Unicode auxiliary plane.

4.1.4 Parameter rules

- 1) Whether the request is successful is independent of the order of request parameters
- 2) Whether the request is successful is independent of the order of occurrence of key-value pairs in the request JSON
- 3) At the time of response processing, it is required not to assume the order of occurrence of key-value pairs in the response JSON
- 4) The new API version may add new parameters or JSON key-value pairs.
- 5) The new API version will not remove existing parameters or JSON key-value pairs
- 6) The data in the API response may contain the data entered by the merchant, i.e., it may be the user input not checked. To avoid XSS (Cross-site scripting) attack, the caller should escape or filter the response data based on the scenario before using it.

4.1.5 Response language

The REST API allows the caller to declare the natural language used for the error description in the response. If necessary, set the HTTP header Accept-Language of the request. Currently support languages:

Language code	Language name
en-US	English

When it is not set or the value is not supported, English (en-US) will be used.

4.1.6 Unique identifier of the request

Each received request is assigned with a unique identifier. The unique identifier of the request is contained in the HTTP header *Customer-Request-Id* of the response. When you need assistance, please provide the unique identifier of the request so that we can locate the specific request more quickly.

4.2 Interface security

The system will provide the merchant with a *key string*, which is used to sign the message; the signature is used to verify the integrity and authenticity of the message.

4.2.1 JWE

Format of JWE protocol header

Parameter	Option	Description	Enumerated Value
alg	M	Algorithm used to encrypt the key. (Asymmetric algorithm)	RSA-OAEP-256 Fixed value
enc	M	Algorithm used to encrypt the confidential fields. (Symmetric algorithm)	A256CBC-HS512 Fixed value

Examples are as follows:

```
{ "alg": "RSA-OAEP-256", "enc": "A256CBC-HS512" }
```

1) JWE algorithm steps

JWE consists of five parts:



The specific generation steps are:

- JOSE has the same meaning as the JWS header.
- Generate a random Content Encryption Key (CEK).
- With the RSA-OAEP-256 encryption algorithm, encrypt the CEK with the public key, and generate the JWE Encrypted Key.
- Generate the JWE initialization vector.
- With the A256CBC-HS512 encryption algorithm, encrypt the plaintext part to generate the Ciphertext; the algorithm will then generate a 256-bit Authentication Tag.
- Perform base64 encoding on the five parts respectively.

2) Java example for JWE encryption

```
public static String jweEncryption(String publicKey, String payload){
    logger.info("Payload before encryption :: {}", payload);
    if (StringUtils.isBlank(publicKey)) {
        Logger.error ("Lack of signed public key");
        throw new
        BusinessException(BaseErrorTable.ENCYR_OR_DECRY_ERROR.getErrorCode(), "Lack of
        verified signed public key");
    }
}
```

```

    }
    /* Encryption */
    String serializedJWE = null;
    try {
        X509EncodedKeySpec keySpec = new
X509EncodedKeySpec(Base64.decodeBase64(publicKey));
        PublicKey pubKey = KeyFactory.getInstance("RSA").generatePublic(keySpec);
        JWEHeader.Builder builder = new JWEHeader.Builder(JWEAlgorithm.RSA_OAEP_256,
EncryptionMethod.A256CBC_HS512);
        JWEHeader header = builder.build();
        JWEEncrypter encryptedJWE = new RSAEncrypter((RSAPublicKey) pubKey);
        JWEObjct jweObject = new JWEObjct(header, new Payload(payload));
        jweObject.encrypt(encryptedJWE);
        serializedJWE = jweObject.serialize();
    } catch (Exception e) {
        logger.error("JWE encryption exception:", e);
        throw new BusinessException(BaseErrorTable.ENCYR_OR_DECRY_ERROR);
    }
    return serializedJWE;
}

```

3) Java example for JWE decryption

```

public static String jweDecryption(String privateKey, String context) {
    logger.info("Context before decryption :: {} ", context);
    if (StringUtils.isBlank(privateKey)) {
        logger.error("Lack of signed private key");
        throw new
BusinessException(BaseErrorTable.ENCYR_OR_DECRY_ERROR.getErrorCode(), " Lack of
signed private key");
    }
    Payload payload = null;
    try {
        PKCS8EncodedKeySpec priPKCS8 = new
PKCS8EncodedKeySpec(Base64.decodeBase64(privateKey));
        PrivateKey priKey = KeyFactory.getInstance("RSA").generatePrivate(priPKCS8);
        JWEDecrypter decryptedJWE = new RSADecrypter((RSAPrivateKey) priKey);
        JWEObjct jweObject = JWEObjct.parse(context);
        jweObject.decrypt(decryptedJWE);
        payload = jweObject.getPayload();
    } catch (Exception e) {
        logger.error("JWE decryption exception:", e);
        throw new BusinessException(BaseErrorTable.ENCYR_OR_DECRY_ERROR);
    }
    return payload.toString();
}

```

4.2.2 JWS

1) Format of JWS protocol header

Parameter	Option	Description	Enumerated value
alg	M	Algorithm used to encrypt the key. (Asymmetric algorithm)	RS256 fixed value
timestamp	M	Timestamp when requested	1548468610

Examples are as follows: {

```
"alg": "RS256",
"timestamp": "1562232686"
}
```

2) JWS algorithm steps

```
// Create the request header
String jWSHeader= new JSONObject().fluentPut("alg", "RS256").fluentPut("timestamp",
System.currentTimeMillis() / 1000 + "").toString();

// Step 1: Compress the jwsheader
String jwsHeaderBase64 =
Base64.encodeBase64URLSafeString(jWSHeader.getBytes("UTF-8"));

// Step 2: Compress the jws payload
String jwsPayloadBase64 = Base64.encodeBase64URLSafeString(reqBody.getBytes("UTF-8"));

// Step 3: Obtain the signature before countersigning
String sign = jwsHeaderBase64 + '.' + jwsPayloadBase64;

// Step 4: Use SHA256 to countersign and obtain the signature after countersigning
String signBySHA256 = signBySHA256WithRSA(sign, privateKey);

// Step 5: Obtain the final jws signature
String jws = jwsHeaderBase64 + ".." + signBySHA256;
```

4.2.3 SHA256WithRSA signature

1. Java example for generation of rsa signature

```
private static String signBySHA256WithRSA(String content, String privateKey) {
    try {
        // Process the private key
        PKCS8EncodedKeySpec priPKCS8 = new
        PKCS8EncodedKeySpec(Base64.decodeBase64(privateKey));

        PrivateKey priKey = KeyFactory.getInstance("RSA").generatePrivate(priPKCS8);

        // Encrypt
        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initSign(priKey);
```

```

signature.update(content.getBytes("UTF-8"));
return Base64.encodeBase64URLSafeString(signature.sign());
} catch (Exception e) {
logger.error("rsa signature generation exception:", e);
throw new BusinessException(BaseErrorTable.SGIN_ERROR);
}
}

```

2. Java example for verification of rsa signature

```

public static boolean verifyJwsBySHA256WithRSA(String content, String sign, String publicKey)
{
    if (StringUtils.isBlank(publicKey)) {
        Logger.error ("Lack of signed public key");
        throw new BusinessException(BaseErrorTable.SGIN_ERROR.getErrorCode(), "lack of signed public key");
    }
    try {
        // Process the public key
        X509EncodedKeySpec keySpec = new
X509EncodedKeySpec(Base64.decodeBase64(publicKey));
        PublicKey pubKey = KeyFactory.getInstance("RSA").generatePublic(keySpec);
        // Decrypt
        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initVerify(pubKey);
        // Specially process the UnionPay jws signature
        String jwsHeaderBase64 = sign.substring(0, sign.indexOf(".."));
        byte[] test = Base64.decodeBase64(jwsHeaderBase64);
        String testStr = new String(test);
        String jwsSignBase64 = sign.substring(sign.indexOf("..") + 2);
        String dateToSign = jwsHeaderBase64 + '.'
        + Base64.encodeBase64URLSafeString(content.getBytes("UTF-8"));
        // Verify that the signature matches the message body
        signature.update(dateToSign.getBytes("UTF-8"));
        return signature.verify(Base64.decodeBase64(jwsSignBase64));
    } catch (Exception e) {
        logger.error("Signature verification exception:", e);
        throw new BusinessException(BaseErrorTable.SGIN_ERROR);
    }
}

```

4.3 Public parameters

4.3.1 HEADER (request header)

Name	Option	Type	Sample Value	Description
Authorization	M	String(1024)	{code}	JWS signature

				string:
Merchant-Id	M	String(12)	7894561	Assigned after the merchant ID accesses the platform
Customer-Request-Id	M	String(64)	12ddfyhdytygbft	Merchant's unique request number
Accept-Language	M	String(8)	en-US	Language

4.3.2 HEADER (response header)

Name	Option	Typ ^e	Sample Value	Description
Authorization	M	String(1024)	{code}	JWS signature string:
Customer-Request-Id	M	String(64)	12ddfyhdytygbft	Merchant's unique request number
Merchant-Id	M	String(12)	7894561	Assigned after the merchant ID accesses the platform

4.4 Interface domain name

Prod Gateway URL: <https://paymentapi.wepayez.com>

4.5 Request example

```

1.  ----- Request -----
2.  POST https://paymentapi.wepayez.com/gateway/payment/cashier/v1/payment
3.  Content-Type: application/json
4.  Accept: application/json
5.  Customer-Request-Id: 15484686101223333
6.  Merchant-Id: 156484566632
7.  Authorization:
    9750ada9a3231.a45308b3c89ad946846ae13c6a734d4f8034c3150cf3e1f99d4
8.  Accept-Language: en-US
9.  {
10.  "billingInformation": {
11.  "city": "xxx1xxx",
12.  "coun@": "CN",
13.  "postcode": "999999",
14.  "state": "xxxx",
15.  "street1": "XXXXXX"

```

AUB Card Gateway API Guide

```
16. },
17. "card":{
18.   "expirationMonth":"02",
19.   "expirationYear":" 2023",
20.   "pan":"420000000000000000",
21.   "securityCode":"123"
22. },
23. "customerInformation": {
24.   "browserInformation": {
25.     "acceptHeader":"html/text",
26.     "challengeWindow" :0,
27.     "javaEnabled":false,
28.     "language":"de",
29.     "screenColorDepth" :0,
30.     "screenHeight": 1200,
31.     "screenWidth":1600,
32.     "timezone":60,
33.     "userAgent":"Mozilla/4.0 (MSIE 6.0; WindowsNT5.0)"
34.   },
35.   "firstName":"jeny",
36.   "lastName":"tom",
37.   "phone":"1234567891245"
38. },
39. "orderInformation": {
40.   "amount":10000,
41.   "attach":"attach",
42.   "goodsDetail": "goodsDetail",
43.   "orderId":"2020020asd30050044"
44. }
45. }
46.
47. ----- Responses -----
48.
49. HTTP/1.1 200
50. Content-Type: application/json
51. Authorization: 9750ada9a3231.a45308b3c89ad946846ae13c6a734d4f8034c3150cf3e1f99d4
52. Customer-Request-Id: 15484686101223333
53. Date: Thu, 29 Oct 2020 09:13:41 GMT
54. Keep-Alive: timeout=60
55. Connection: keep-alive
56.
57. {
58.   "code": "99",
59.   "message": "System error"
60. }
```

5 Cashier Interface

5.1 H5 Payment

5.1.1 Description

The merchant calls the cashier to place an order, allowing the cardholder to select the corresponding card type to make payment.

5.1.2 Address

Prod Request URL	https://paymentapi.wepayez.com/gateway/payment/cashier/v1/payment
Request mode	POST

5.1.3 Request

Name	Option	Type	Sample Value	Description
orderInformation	M	JSON		Order information

orderInformation

Name	Option	Type	Sample Value	Description
amount	M	int	9200	Transaction amount, transaction amount = commodity amount + tip + additional amount
orderId	M	String(64)	2020020asd30050044	Merchant Serial Number (MSN)
goodsDetail	O	String(128)	Good	Commodity information
attach	O	String(128)	Good	Information attached by the merchant; returned as it is in the response information
callbackUrl	C	String(2048)	http://demo.xx.com	The page that is redirected to after successful payment by the cardholder
notifyUrl	C	String(2048)	http://demo.notify.com	The URL to which a notification to the merchant is sent after successful payment by the cardholder

service	O	String(128)		Payment type
operatorId	O	String(32)	4321112	Operator ID
operatorName	O	String(32)	Toney	Operator name
orderAmount	O	int	9200	Commodity amount
tipFee	O	int	10	Tip
surcharge	O	Int	300	Additional amount
validityPeriod	O	Int(5)	1	Validity(in minutes). Default: 10 mins.

5.1.4 Response

Name	Option	Type	Sample Value	Description
code	M	String(2)	00	Response code
message	M	String(64)	Approved	Describe the information of status code
data	C	JSON		When the code is 00, this value will be returned
When the code is 00, the following fields will be returned in data				
orderinformation	M	JSON		Order information
cashierUrl	M	String(2048)	http://cashierUrl.com	The merchant can use this link to redirect to the cashier

orderinformation

Name	Option	Type	Sample Value	Description
orderId	M	String(64)	2020020asd30050044	Merchant Serial Number (MSN)
goodsDetail	O	String(128)	Good	Commodity information
attach	O	String(128)	Good	Information attached by the merchant
currency	M	String(3)	USD	Currency
amount	M	int	9800	Transaction amount, transaction amount = commodity amount + tip + additional amount
paymentType	M	String(2)	PAY	Type of order PAY, payment
responseDate	M	Date	2020-10-08T10:34:56+08:00	Response date

service	O	String(128)		Payment type
operatorId	O	String(32)	4321112	Operator ID
operatorName	O	String(32)	Toney	Operator name
orderAmount	O	int	9200	Commodity amount
tipFee	O	int	10	Tip
surcharge	O	Int	300	Additional amount

5.2 Query

5.2.1 Description

The merchant calls this interface to get the status of the cashier order

5.2.2 Address

Pro Request URL	https://paymentapi.wepayez.com/gateway/payment/cashier/v1/inquiry
Request mode	POST

5.2.3 Request

Name	Option	Type	Sample Value	Description
orderInformation	M	JSON		Order information

orderInformation

Name	Option	Type	Sample Value	Description
orderId	M	String(64)	2020020asd30 050044	Merchant Serial Number (MSN)

Response

5.2.4

Name	Option	Type	Sample Value	Description
code	M	String(2)	00	Response code
message	M	String(64)	Approved	Describe the information of status code
data	C	JSON		When the code is 00, this value will be returned
When the code is 00, the following fields will be returned in data				
orderInformation	M	JSON		Order information
card	M	JSON		Card information

orderInformation

AUB Card Gateway API Guide

Name	Option	Type	Sample Value	Description
orderId	M	String(64)	2020020asd30050044	Merchant Serial Number (MSN)
goodsDetail	O	String(128)	Good	Commodity information
attach	O	String(128)	Good	Information attached by the merchant
currency	M	String(3)	USD	Currency
amount	M	Int	9200	Transaction amount, transaction amount = commodity amount + tip + additional amount
responseDate	M	Date	2020-10-08T10:34:56+08:00	Response date
transactionResult	M	String(10)	SUCCESS	States of transaction 1) <i>PENDING</i> : unknown 2) <i>SUCCESS</i> : successful 3) <i>FAILED</i> : failed
paymentType	M	String(2)	PAY	Type of order 1) <i>REFUND</i> : refund 2) <i>PAY</i> : payment
orderAmount	C	BigDecimal(12)	9800	Commodity amount
surcharge	O	Int	300	Additional amount
tipFee	O	BigDecimal(12)	300	Tip
service	O	String(128)		Payment type
operatorId	O	String(32)	4321112	Operator ID
operatorName	O	String(32)	Toney	Operator name

card

Name	Option	Type	Sample Value	Description
paymentBrand	M	String(16)	VISA	The card organization that this card belongs to
cardBin	M	String(6)	420000	First 6 digits of the card number
last4Digits	M	String(4)	0000	Last 4 digits of the card number

5.3 Refund

5.3.1 Description

The merchant calls this API to refund the cashier order

5.3.2 Address

Prod Request URL	https://paymentapi.wepayez.com/gateway/payment/cashier/v1/refund
Request mode	POST

5.3.3 Request

Name	Option	Type	Sample Value	Description
orderinformation	M	JSON		Order information

orderinformation

Name	Option	Type	Sample Value	Description
amount	M	int	9200	Order amount, displayed in minimum currency unit
orderId	M	String(64)	2020020asd30050044	Merchant Serial Number (MSN)
originalReferencedId	M	String(64)	W202101022224488888	Wallyt serial number

5.3.4 Response

Name	Option	Type	Sample Value	Description
code	M	String(2)	S0000	Response code
message	M	String(64)	Approved	Describe the information of status code
data	C	JSON		When the code is 00, this value will be returned

When the code is 00, the following fields will be returned in data				
orderinformation	M	JSON		Order information
card	M	JSON		Card information

orderinformation

Name	Option	Type	Sample Value	Description
referencedId	M	String(64)	W2021010222 24488888	Wallyt serial number
originalReferencedId	M	String(64)	W2021010222 24488888	Original Wallyt serial number
originalOrderId	C	String(64)	2020020asd30 050044	When the order type queried is refund, the pre-authorization is completed, and the interface is cancelled, the changed field returns the original transaction order serial number.
orderId	M	String(64)	2020020asd30 050044	Merchant Serial Number (MSN)
currency	M	String(3)	USD	Currency
amount	M	int	9200	Refund amount
paymentType	M	String(2)	REFUND	Type of order <i>REFUND</i> : refund
responseDate	M	Date	2020-10-08T1 0:34:56+08:00	Response date
operatorId	O	String(32)	4321112	Operator ID
operatorName	O	String(32)	Toney	Operator name
transactionResult	M	String(10)	SUCCESS	States of transaction 1) <i>PENDING</i> : unknown 2) <i>SUCCESS</i> : successful 3) <i>FAILED</i> : failed
service	O	String(128)		Payment type

card

Name	Option	Type	Sample Value	Description
paymentBrand	M	String(16)	VISA	The card organization that this card belongs to
cardBin	M	String(6)	420000	First 6 digits of the card number
last4Digits	M	String(4)	0000	Last 4 digits of the card number

5.4 Notification**5.4.1 Description**

After the cardholder pays the cashier order, the result can be received through this API.

5.4.2 Notification Request

Name	Option	Type	Sample Value	Description
code	M	String(2)	00	Response code
data	C	JSON		When the code is 00, this value will be returned
message	M	String(64)	Success	Describe the information of status code

When the code is 00, the following fields will be returned in data				
orderinformation	M	JSON		Order information
card	M	JSON		Card information

orderinformation

Name	Option	Type	Sample Value	Description
referencedId	M	String(64)	W202101022224488888	Wallyt serial number
orderId	M	String(64)	2020020asd30050044	Merchant Serial Number (MSN)
goodsDetail	O	String(128)	Good	Commodity information
attach	O	String(128)	Good	Information attached by the merchant
orderAmount	C	BigDecimal(12)	9800	Commodity amount
surcharge	O	Int	300	Additional amount

tipFee	O	BigDecimal (12)	300	Tip
currency	M	String(3)	USD	Currency
amount	M	int	9200	Transaction amount, transaction amount = commodity amount + tip + additional amount
responseDate	M	Date	2020-10-08T10:34:56+08:00	Response date
transactionResult	M	String(10)	SUCCESS	States of transaction 1) SUCCESS: successful 2) FAILED: failed
paymentType	M	String(2)	PAY	Type of order 1) PAY: payment
operatorId	O	String(32)	653204	Operator ID
qrTag	O	int	1	Dynamic code identification 1) Dynamic code 2) Fixed code
paymentBrand	O	String(16)	MasterCard	Payment brand MasterCard

5.4.3 Merchant Response

The merchant needs to return the text SUCCESS in the response body to indicate that the result notification has been received.

6. Appendix

6.1 Error code

Name	Description	Reason	Solution	Interface
00	SUCCESS	Transaction successful		All
01	3DS TRANSACTION RESULTUNKNOWN, WAITING TO RECEIVE THE NOTICE TO INQUIRE AGAIN	3ds transaction result is unknown, waiting for notification for another query		All

AUB Card Gateway API Guide

02	MERCHANT VERIFIES RELEVANT ERRORS	Merchant check related error		All
03	ROUTING INFORMATION WAS MISCONFIGURED	Routing information configuration error		All
04	MERCHANT DOES NOT CONFIGURE THIS PAYMENT TYPE	The merchant did not configure the payment type		All
05	ORDER INFORMATION DOES NOT EXIST	Order information does not exist		All
06	SGIN ERROR	Signature error		All
07	ENCRYPTION OR DECRYPTION ERROR	Encryption and decryption error		All
08	DOUBLE REPORTING OF MERCHANTS	Duplicate reporting to the merchant		All
09	PARAMETER IS EMPTY OR MALFORMED	The parameter is empty or the format is wrong		All
10	[Customer-Request-Id] IS EXIST	The request serial number already exists		All
11	SERVICE DEMOTION	Service degradation		All
12	THE REQUEST IS FREQUENT	Frequent requests		All
20	REFER TO THE CARD ORGANIZATION THAT THIS CARD BELONGS TO	Please contact the card issuing bank		All
21	INVALID MERCHANT	Invalid merchant		All
22	PLEASE GET CARD INFORMATION	Please get card information		All

AUB Card Gateway API Guide

23	DO NOT HONOR	Unfulfilled transaction		All
24	INVALID TRANSACTION	Invalid transaction		All
25	INVALID AMOUNT	Invalid order amount		All
26	PLEASE CHECK THE CARD NUMBER INFORMATION	Please check the card number information		All
27	PLEASE CHECK THE CARD NUMBER INFORMATION	The request serial number already exists		All
28	THE CARD IS A LOST CARD AND CANNOT BE TRADED	This card is a lost card and cannot be used for transaction		All
29	THIS CARD HAS BEEN LOCKED AND CANNOT BE TRADED	This card has been locked and cannot be used for transaction		All
30	INSUFFICIENT CARD LIMIT	Card limit insufficient		All
31	CURRENT CARD HAS EXPIRED	Current card expired		All
32	INVALID PIN	Invalid PIN		All
33	TRANSACTION NOT PERMITTED TO ISSUER/CARDHOLDER	Issuing bank/cardholder not allowed to make transactions		All
34	TRANSACTION NOT PERMITTED TO ACQUIRER	Acquirer not allowed to make transactions		All
35	EXCEEDS WITHDRAWAL AMOUNT LIMIT	Withdrawal limit exceeded		All
36	THIS CARD HAS BEEN RESTRICTED AND CANNOT BE TRADED	This card has been restricted and cannot be used for transaction		All
37	VIOLATION OF SECURITY REGULATIONS, UNABLE TO TRADE	It cannot be used for transaction due to violation of security regulations		All

AUB Card Gateway API Guide

38	EXCEEDS WITHDRAWAL COUNT LIMIT	Limit of withdrawal times exceeded		All
39	REFER TO THE CARD ORGANIZATION THAT THIS CARD BELONGS TO	Please contact the card issuing bank		All
40	PLEASE MODIFY THE PIN	Please modify the card password		All
41	ALLOWABLE NUMBER OF PIN TRIES EXCEEDED	Number of incorrect password entries exceeded the limit		All
42	DOMESTIC DEBIT TRANSACTION NOT ALLOWED	Domestic debit transactions not allowed		All
43	UNABLE TO VERIFY PIN	Unable to verify password, please try again later		All
44	WRONGPIN,PLEASE RE-ENTER	Wrong password, please re-enter		All
45	AUTHORIZATION SYSTEM ERROR, PLEASE CONTACT THE ADMINISTRATOR	Authorization system error, please contact the administrator		All
46	SYSTEM ERROR, TRANSACTION FAILED	System error, transaction failed		All
47	DUPLICATE TRANSMISSION DETECTED	Order number already exists		All
96	TRANSACTION UNKNOWN	Unknown transaction result		All
97	TRANSACTION TIMEOUT	Transaction timeout		All
98	TRANSACTION FAILURE	Transaction failed		All
99	SYSTEM ERROR	System busy		All